VIET NAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



MULTIDISCIPLINARY PROJECT

SIMPLE ROBOT USING LIDAR AND RASPBERRY PI WITH AI APPLICATION

MAJORS: COMPUTER SCIENCE & COMPUTER ENGINEERING

COUNCIL: CSE
INSTRUCTOR: PHẠM HOÀNG ANH, PhD.

Student 1: Lê Hoàng Mai Phương
Student 2: Nguyễn Việt Phương
Student 3: Hồ Nguyễn Ngọc Bảo
Student 4: Nguyễn Trấn Quốc

Ho Chi Minh City, April 2022

Table of contents

Chapter 1. INTRODUCTION **2**

1.1 Robot structure 2

1.2 Introduction of the main devices 5

Chapter 2. PROCESSES **7**

2.1 Setting Raspberry Pi, Motor Driver board, and control the robot's movements 7

2.1.1 Setting Raspberry Pi and Operating system 7

2.1.2 Setting the Motor Driver to control the robot and publishing the **/cmd_vel** topic 8

2.1.3 Result 11

2.2 Set up LiDAR and process LiDAR's data 12

2.3 Set up the transformation for the robot 12

2.3.1 Theory on transformation and writing the URDF/X-acro file describing the robot structure 12

2.3.2 Publishing the **/tf** topic using the Robot State Publisher 14

2.4 Set up wheel encoder sensor and odometry 15

2.4.1 Publishing ticks count topic 15

2.4.2 Calculating and publishing **/odom** topic 17

2.5 Mapping 19

2.5.1 Hector mapping 20

2.5.2 gmapping 21

2.5.3 Difference between Hector and gmapping 22

2.5.4 Mapping setup 23

2.6 Localization 25

2.6.1 Reinforcement Learning and Monte Carlo 25

2.6.2 AMCL algorithm 26

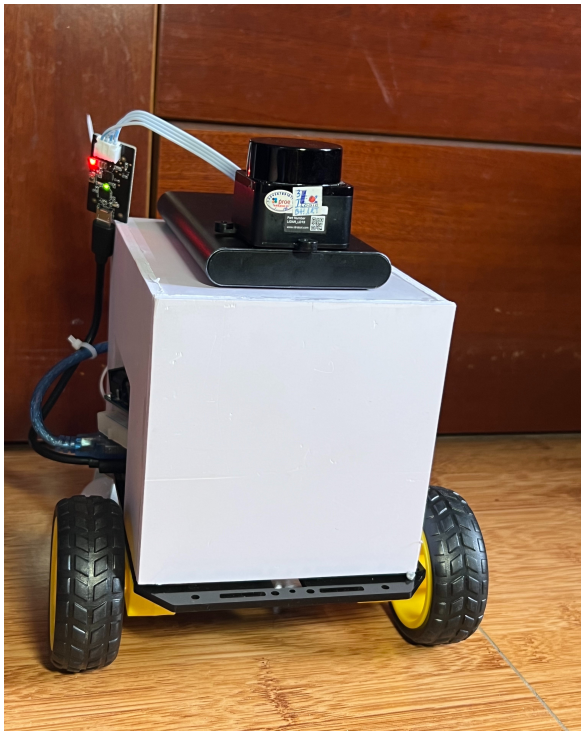
2.6.3 AMCL setup 27
2.7 Move Base 28
2.8 Our Code 28

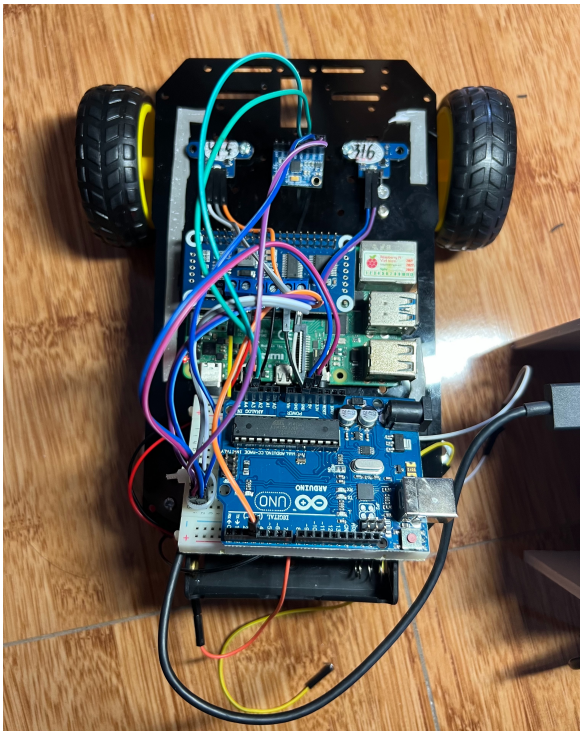
REFERENCES **29**

CHAPTER 1

INTRODUCTION

1.1 Robot structure





- Robot chassis: a mica acrylic sheet size 198 x 118 x 3mm.
- Wheels: two 65mm plastic wheels at front, two castor wheels at rear.

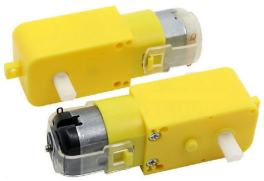




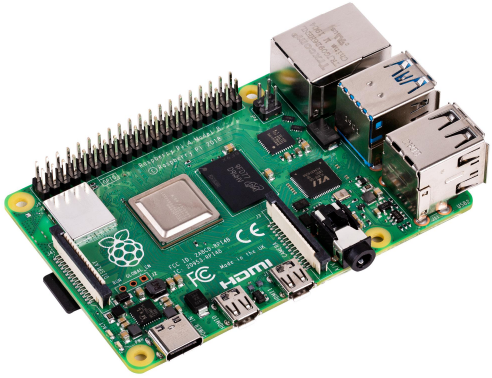
- Speed sensor Encoder V2: two sensors connect to two wheels to count ticks and publish data to Arduino.



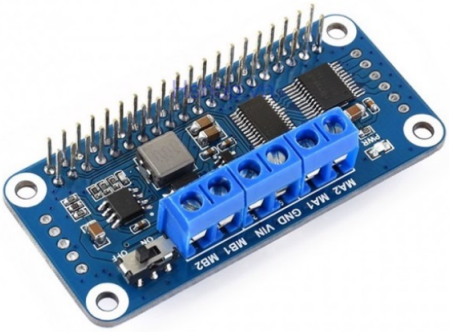
- Gear Motor DC V1: two motor connect to two front wheels and Motor Driver board.

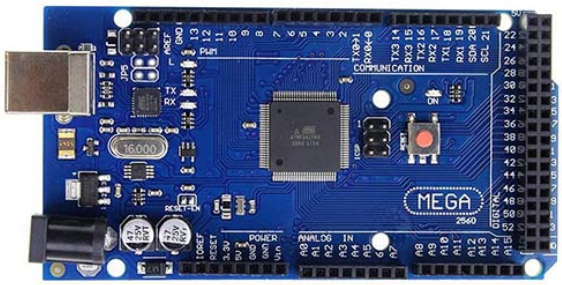


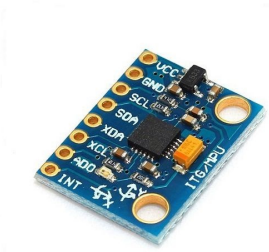
- Raspberry Pi: the computer to control the whole system. Connect to the power source, Motor Driver board, Arduino board, and LiDAR.



- Motor Driver board: to control and communicate between wheels and Raspberry Pi.
- Arduino mega 2560 CH340: connection between two speed sensors, IMU, and Raspberry Pi and publishing the topic for Odometry.
- IMU sensor: Connect to Arduino to send information about the robot's Gyro and Accelerometer.
- LiDAR: a 360° LiDAR on the robot's roof, connects to Raspberry Pi.
- Power bank: A 10.000mah power bank connects to Raspberry Pi as power source.


M2 M1 GND VIN M1 M2


MEGA
2560
COMMUNICATION
POWER
ANALOG IN


VCC
GND
SCL
SDA
XDA
XCL
ADO
INT



- Battery: two rechargeable batteries connect to Motor Driver board to power the front wheels.

1.2 Introduction of the main devices

1. **DTOF LiDAR LD19**: DTOF Laser Ranging Sensor, 360° Omni-Directional Lidar, UART Bus with:
   - Ranging distance: 0.02 $\sim$ 12.00 m
   - Ranging frequency: 4500 Hz
   - Scanning frequency: 5 $\sim$ 13 Hz
   - Ranging average frequency: $\pm$ 45 mm (within 0.3 $\sim$ 12.00m)
   - Scanning angle: 360°
2. **Raspberry Pi 4 Model B**: A Raspberry Pi computer with 2GB/RAM to link and control the entire robot system. About the operating system, we used **Raspbian Buster** with **ROS 1 Noetic** for our Raspberry Pi, and **Ubuntu 20.04 LTS** with **ROS 1 Noetic** on our computer to communicate with the Raspberry Pi.
3. **Waveshare Motor Driver HAT for Raspberry Pi**: The Motor Driver designed for Raspberry Pi, controlled via I2C interface, will drive two DC motors at the same time using the Driver TB6612FNG with a maximum capacity 12VDC/1.2A. We connect the Motor Driver to the Raspberry Pi (using an IC PCA9685 node) and to the two front wheels of the robot (using the protected pins to publish PWM). It's specifications:
   - Model: Waveshare Motor Driver HAT for Raspberry Pi, I2C Interface
   - Power supply: 6 12VDC (VIN terminal)
   - Logic voltage: 3.3VDC
   - Standard Raspberry Pi 40PIN GPIO extension header, supports Raspberry Pi series boards
   - I2C controlled, by setting 5 address jumpers, it is stackable up to 32 this modules
   - Onboard PCA9685 chip, provides 12-bit hardware PWM to adjust

motor speed

- Onboard TB6612FNG dual H-bridge motor driver, high efficiency, low heating
- Integrates 5VDC regulator, up to 3A output current, can be powered from battery through VIN terminal
- Reserved I2C control pins, allows to work with other control boards

CHAPTER 2

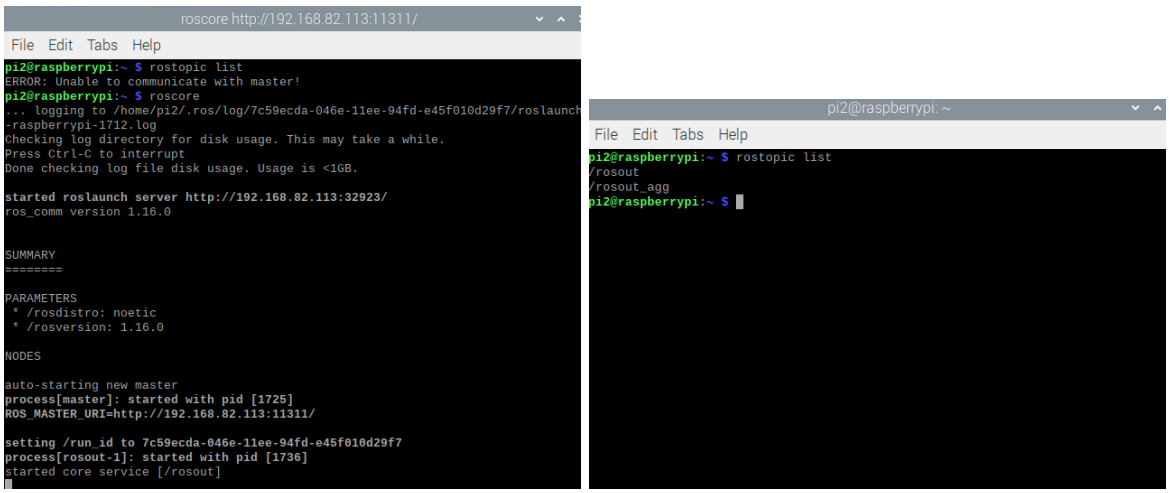
PROCESSES

For this project, we need the data of these four topics: **/cmd_vel**, **/scan**, **/tf**, **/odom** to control and mapping by using our robot. Below, we will describe our process in detail and all the data we receive from it.

2.1 Setting Raspberry Pi, Motor Driver board, and control the robot's movements

2.1.1 Setting Raspberry Pi and Operating system

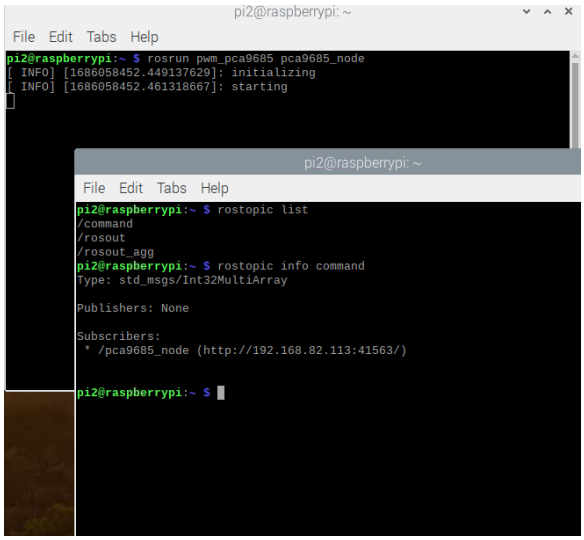
After some research on ROS distribution, we decided to use ROS 1 Noetic for our Raspberry Pi 4 running Raspbian Buster.
To install the OS, we simply download the image, specifically Raspbian Buster 32-bit image, from Raspberry Pi's official website and write the image onto an empty SD card using Raspberry Pi Imager.
As for ROS Noetic's installation, we used this tutorial.
As Raspberry Pi 4 has limited processing power, we also installed ROS Noetic onto a separate PC running Ubuntu 20.04 and configured the Raspberry Pi as the ROS master (as a SBC) so the PC can help with visualization and mapping.
Below are the photos of all rostopics we have before and after we run the command **roscore**. If topics **/rosout** and **/rosout_agg** appear on the screen, then we have run the command successfully. This ROS master node will handle all the communications of the nodes for us. Next time, whenever we turn on the SBC, we have to dedicate the first terminal for this ROS master node.



2.1.2 Setting the Motor Driver to control the robot and publishing the /cmd_vel topic

2.1.2.1 Publishing the node to handle the PWM sending form pins to motors

We linked the Motor Driver board to our Raspberry Pi, and also connected it to the robot wheels with PIN MA1 and MA2 for gear motor in right wheel, PIN MB1 and MB2 for gear motor in left wheel. Motors are in the front line and two castor wheels in the back line to help the car rotate smoother.
Our motor driver HAT communicates with the Raspberry Pi through the I2C pins so it is enabled first in the settings. For controlling our Motor Driver model, we used this ROS node for the PCA9685 package.
We run the command below to run the PCA node, after that, we will see a new topic name **/command**.



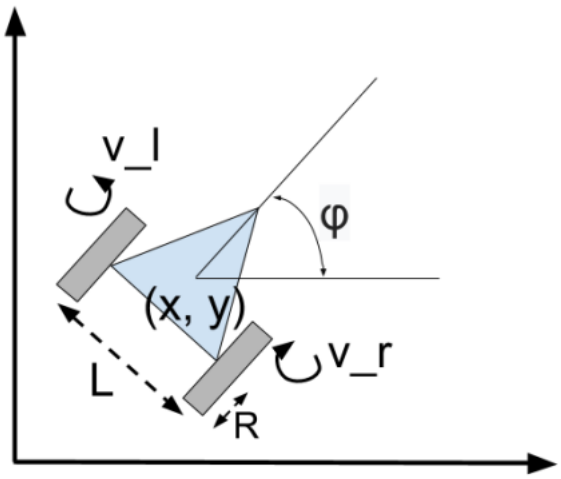
2.1.2.2 Understanding the theory and basic kinematics

We focus on 3 states that define robot's pose:

- x - position on the x-axis (in meters)
- y - position on the y-axis (in meters)
- φ - phi - angle of the unicycle counter clockwise from x-axis (in radians)

For a unicycle model, there are 2 inputs that affect these 3 states of the robot:

- v = forward velocity (in meters per second)
- w = angular velocity (in radians per second)



with:

- v_r = clockwise angular velocity of right wheel (in radians per second)
- v_l = counter-clockwise angular velocity of left wheel (in radians per second)
- L = wheelbase (in meters per radian)
- R = wheel radius (in meters per radian)

We used the kinematics for a unicycle model and kinematics for a differential drive model and following these equations to convert unicycle v and w inputs into v_r and v_l differential drive inputs for our robot:

- v_r = $\frac{2v+wL}{2R}$
- v_l = $\frac{2v-wL}{2R}$

2.1.2.3 Combine the knowledge above and implement the code for the /cmd_vel topic

We set a class name **MotorDriver** in a python file **motor_driver.py** to set all the movements of our robot, such as forward, stop, reverse, turn, and pivot. This file is the main logic to calculate and decide how the car move. The file is in our **test** package.

Then, we have a file named **move_with_cmd_vel.py** to feed the above file with parameters and subscribe to the **/cmd_vel** topic. Usually, we will set the PWM that the driver motor sends to 50% of its maximum capacity and a multiplier standard of 1. Whenever we publish a Twist message to **/cmd_vel**, the robot moves accordingly to the logic.

```
#!/usr/bin/env python3
import rospy
import sys
from geometry_msgs.msg import Twist
from motor_driver import MotorDriver

class RobotMover(object):

    def __init__(self, value_BASE_PWM, value_MULTIPLIER_STANDARD, value_simple_mode):
        rospy.Subscriber("/cmd_vel", Twist, self.cmd_vel_callback)
        self.motor_driver = MotorDriver( i_BASE_PWM=value_BASE_PWM,
                                         i_MULTIPLIER_STANDARD=value_MULTIPLIER_STANDARD,
                                         simple_mode=value_simple_mode)
        rospy.loginfo("RobotMover Started...")


    def cmd_vel_callback(self, msg):
        linear_speed = msg.linear.x
        angular_speed = msg.angular.z

        # Decide Speed
        self.motor_driver.set_cmd_vel(linear_speed, angular_speed)


    def listener(self):
        rospy.spin()

if __name__ == '__main__':
    rospy.init_node('morpheuschair_cmd_vel_listener', anonymous=True)

    if len(sys.argv) > 5:
        value_BASE_PWM = int(float(sys.argv[1]))
        value_MULTIPLIER_STANDARD = float(sys.argv[2])
        value_simple_mode = sys.argv[3] == "true"
```

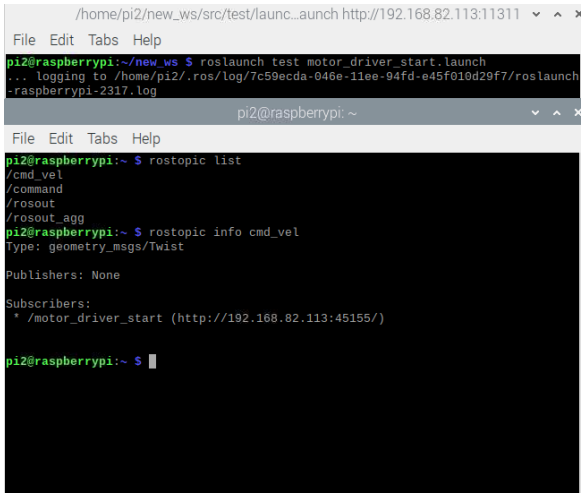
```
35
36          robot_mover = RobotMover(value_BASE_PWM,
37                                   value_MULTIPLIER_STANDARD,
38                                   value_simple_mode)
39          robot_mover.listener()
```

2.1.3 Result

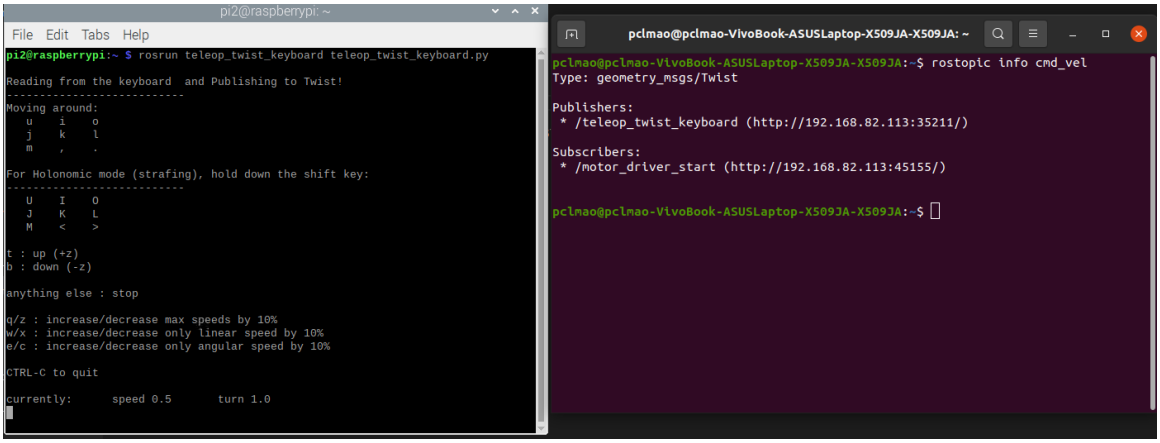
Follow all the above codes and packages, we launch the Motor Driver by using:

```
~$ roslaunch test motor_driver_start.launch
```

Then we list all the topics that we have, we will see a new topic, which is one of the core topics of our project, named **/cmd_vel**.



To control the robot by our computer's keyboard, we can run a python file name **teleop_twist_keyboard.py**, then we will receive this result:



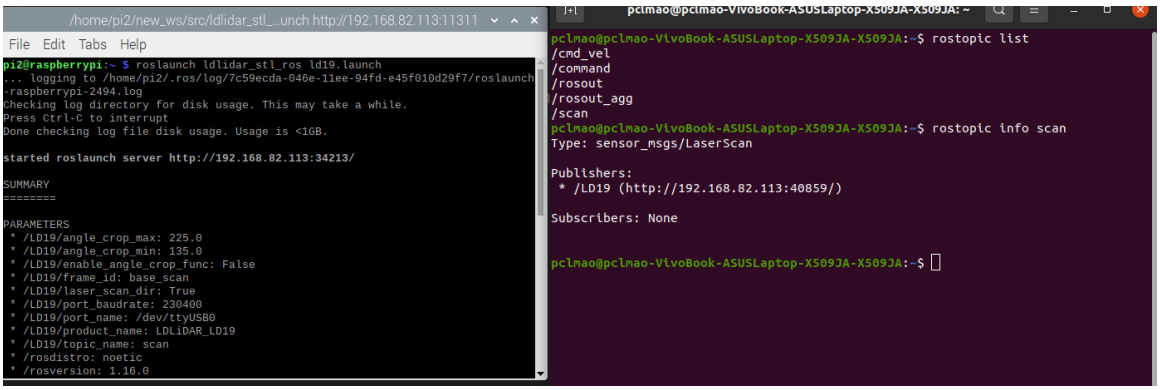
You can test and debug the motor driver while publishing messages on this topic.

2.2 Set up LiDAR and process LiDAR's data

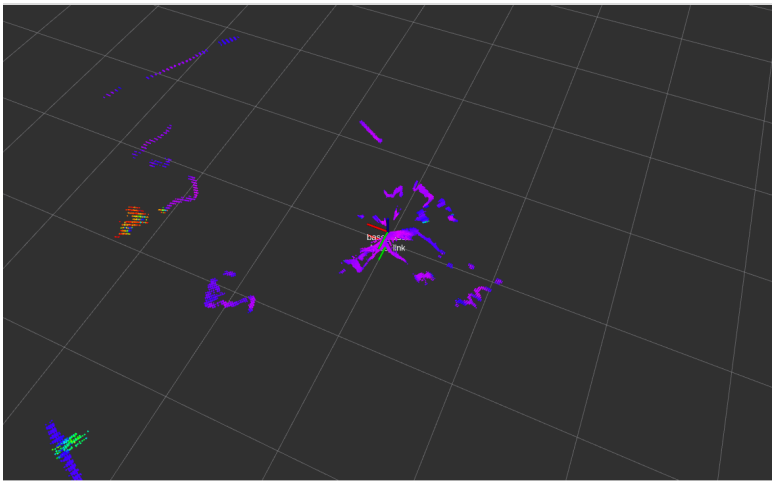
We put the LiDAR on the top of our robot for a better view, then connected it to our Raspberry Pi as power and control source.
After power up, we launch our LiDAR using the launch commend on ROS:

```
~$ roslaunch ldlidar_stl_ros ld19.launch
```

Open a new terminal and check for all the rostopic at present, we will see a new topic name **/scan** from our laser scan of the LiDAR.



Then we open Rviz to see our LiDAR's laser scan in the Rviz map



2.3 Set up the transformation for the robot

2.3.1 Theory on transformation and writing the URDF/Xacro file describing the robot structure

Imagine the following scenario. You have a robot, let's say a TurtleBot3 robot, with a laser mounted on top of it. To use the laser readings, you need

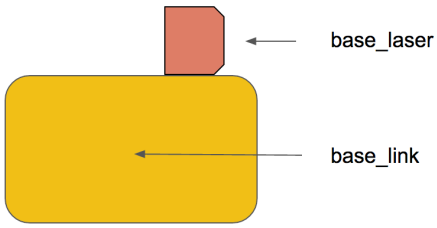
to transform between the laser and the robot base and add it to the transform tree.

Ok, let's clarify all of this. So, you have a TurtleBot3 Robot with a laser mounted on it. But, to use the laser data, you need to tell the robot **WHERE (position and orientation)** this laser is mounted in the robot. This is what is called a transformation between frames.

A transform specifies how data expressed in a frame can be transformed into a different frame. For instance, if you detect an obstacle with a front laser at 3 cm, it is 3 cm from the laser, but not from the center of the robot (that is usually called the **/base_link** or **/base_footprint**). So to know the distance from the center of the robot, transform the 3 cm from the **/laser_frame** to the **/base_link** frame (which is what the Path Planning system needs to know, the distance from the center of the robot to the obstacle).

Don't you think so? If you don't provide this information to the robot when the laser detects an object, how can the robot know where this object is? Is it in front of the robot? Is it behind? Is it to the right? There's no way the robot can know it if you don't tell the robot **the POSITION and the ORIENTATION** of the laser relative to the center of the robot. To do this, do the following:

First, define two frames (coordinate frames), one at the center of the laser and another at the center of the robot. Usually, the center of the robot is placed at the rotational center of the robot. You'll name the laser frame as **/base_laser** and the robot frame as **/base_link**.



Here you can see a scheme of how it would look: Transformation for our robot.
Now, define a relationship (in terms of position and orientation) between the **/base_laser** and the **/base_link**. For instance, you know that the **/base_laser** frame is at a distance of 5 cm on the x-axis and 11 cm on the

z-axis, referring to the **/base_link** frame. Then you'll have to provide this relationship to the robot.

This relationship between the laser's position and the base of the robot is known in ROS as the TRANSFORM between the laser and the robot.

Most transformations are defined in a Xacro or a URDF file, where each of the robot's components is a **link** and they connect via a **joint**. These files are written in XML format. You can read more about different kinds of links and joints that ROS provided here.

.

2.3.2 Publishing the /tf topic using the Robot State Publisher

As above, we provide the transformation of our robot but keeping track of the connection between all the links in real-time would be a headache. Luckily ROS provides us with State Publisher to delegate that task. You can find this in the **rviz.launch** file in the **rosbot** package that we provided.

```
<?xml version="1.0"?>
<launch>

  <param name="robot_description" command="$(find xacro)/xacro '$(find rosbots)/urdf/rosbots.
    xacro'"/>

  <!-- send fake joint values -->
  <node name="joint_state_publisher" pkg="joint_state_publisher" type="joint_state_publisher">
    <param name="use_gui" value="False"/>
  </node>

  <!-- Combine joint values -->
  <node name="robot_state_publisher" pkg="robot_state_publisher" type="robot_state_publisher"/
    >

  <!-- Run LiDAR, you can uncomment this after finishing the above processes-->
  <!-- <include file="$(find ldlidar_stl_ros)/launch/ld19.launch"/> -->

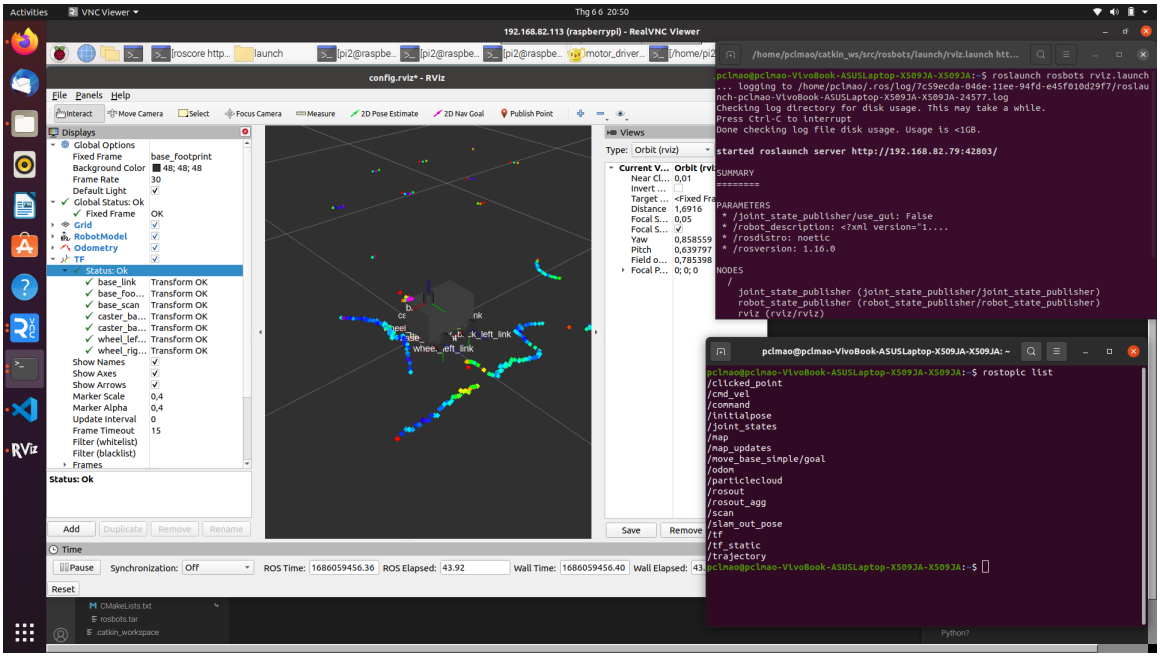
  <!-- Show in Rviz   -->
  <node name="rviz" pkg="rviz" type="rviz"
  args="-d $(find rosbots)/rviz/config.rviz"/>

</launch>
```

Run the below code in your PC to publish the **/tf** topic and debug/visualize our robot on Rviz.

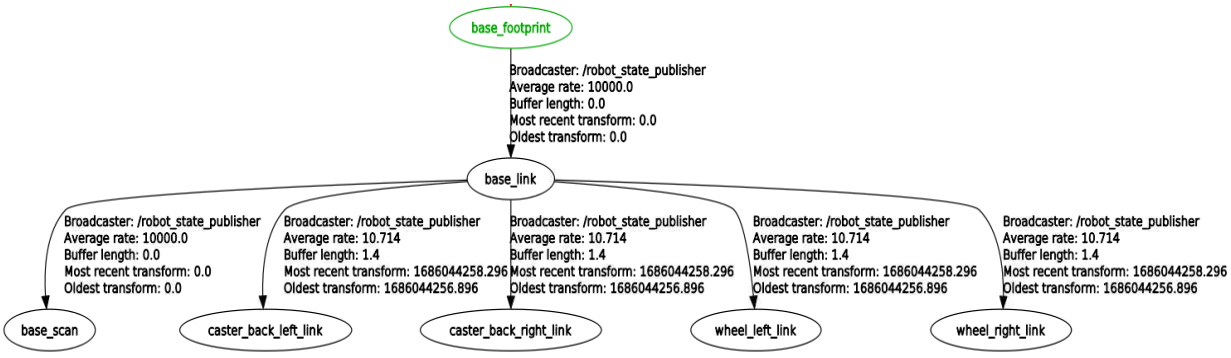
```
~$ roslaunch rosbots rviz.launch
```

Here is the result:



We can also check the transform tree of the robot by typing in your PC:

```
~$ rosrun rqt_tf_tree rqt_tf_tree
```



2.4 Set up wheel encoder sensor and odometry

2.4.1 Publishing ticks count topic

The sensor module we employ is to accurately measure the rotation speed of a wheel by looking at the rate of interruption of the light (in tick). It contains a photo interrupter sensor that emits a beam of light and a receiver

that detects the interruption of light.

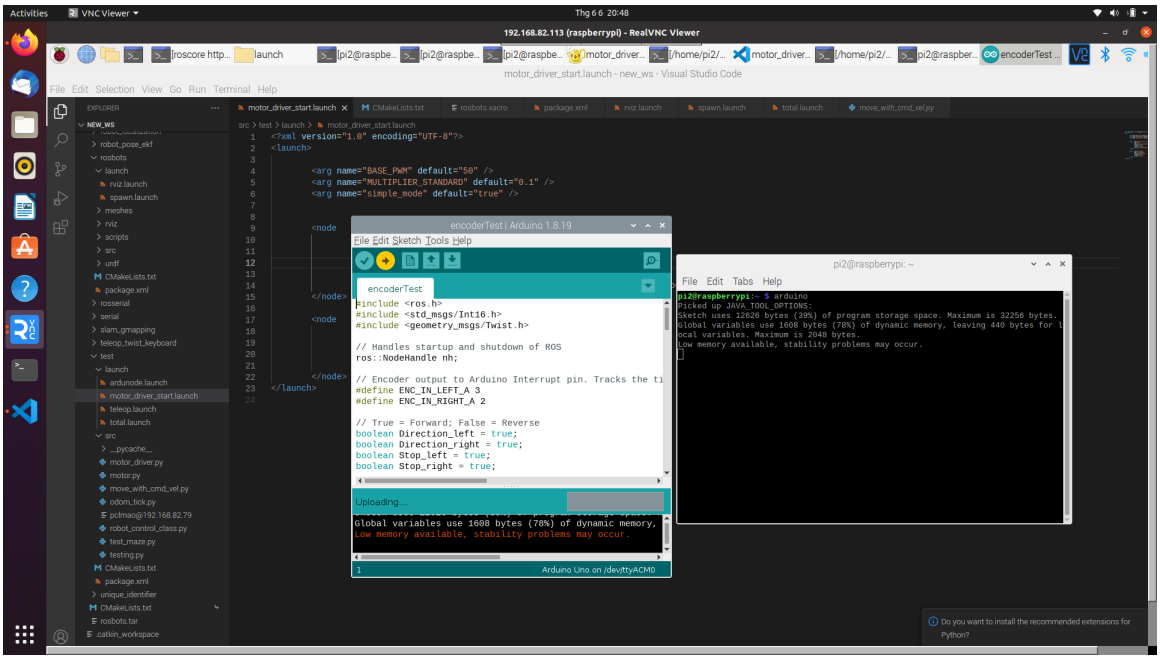
By analyzing the frequency of these interruptions, we can determine the rotation speed of the wheel as data and transfer its data to Arduino UNO R3 to read it.

Hardware setup:

- Connect VCC pin On the Sensor to the 5V pin of the Arduino to provide power to the sensor.
- Connect GND pin of the Sensor to GND pin of the Arduino
- Connect OUT pin to pin 2 and 3 of the Arduino to read the signals.

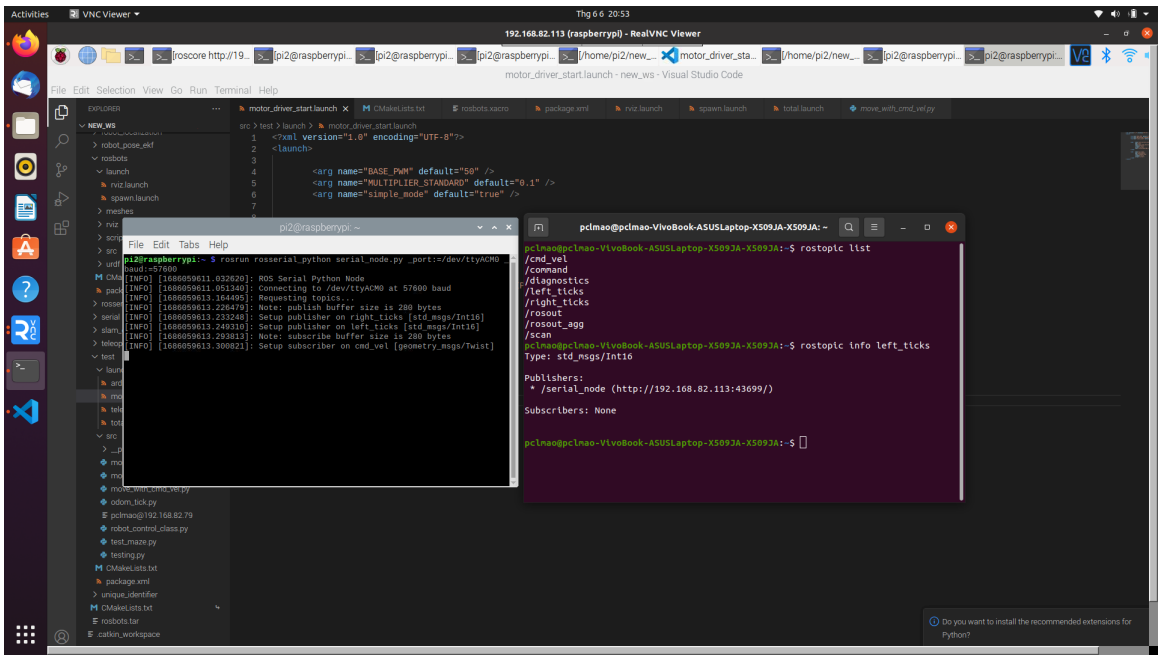
Upload the Arduino code to count the ticks and publish it to the SBC by typing to the SBC:

Firstly, make sure the Arduino code is correct by compiling the sketch in SBC:



Then, let's run the node that connects the Arduino and Raspberry Pi using serial:

```
~$ rosrun rosserial_python serial_node.py _port:=/dev/ttyACM0 _baud:=57600
```



2.4.2 Calculating and publishing /odom topic

Then we run the Arduino code which can count interruptions and publish the ticks count topic into SBC. Based on the frequency of interruptions we can calculate the distance our car travel in the period of time(in tick) by the formula:

$$Ld = (2 \times \pi \times wheelradius) \times (Lt/TPR)$$

With:

- Ld: Total traveled the distance of the left wheel
- wheelradius: Radius of the wheel which is 3.3 cm
- Lt: Total ticks the left wheel have accumulated
- TPR: Ticks per roulette

After calculating the distance, we publish the transform over tf which is responsible for managing the relationships between coordinate frames relevant to the robot in the transform tree.

```
odom_broadcaster.sendTransform(
      (x, y, 0.),
      odom_quat,
      current_time,
      "base_footprint",
      "odom"
   )
```

Then we continue on publish the odometry message over ROS

```
odom = Odometry()
  odom.header.stamp = current_time
  odom.header.frame_id = "odom"

  odom.pose.pose = Pose(Point(x, y, 0.), Quaternion(*odom_quat))

  if dt>0:
     vx=dx/dt
     vy=dy/dt
     vth=dth/dt

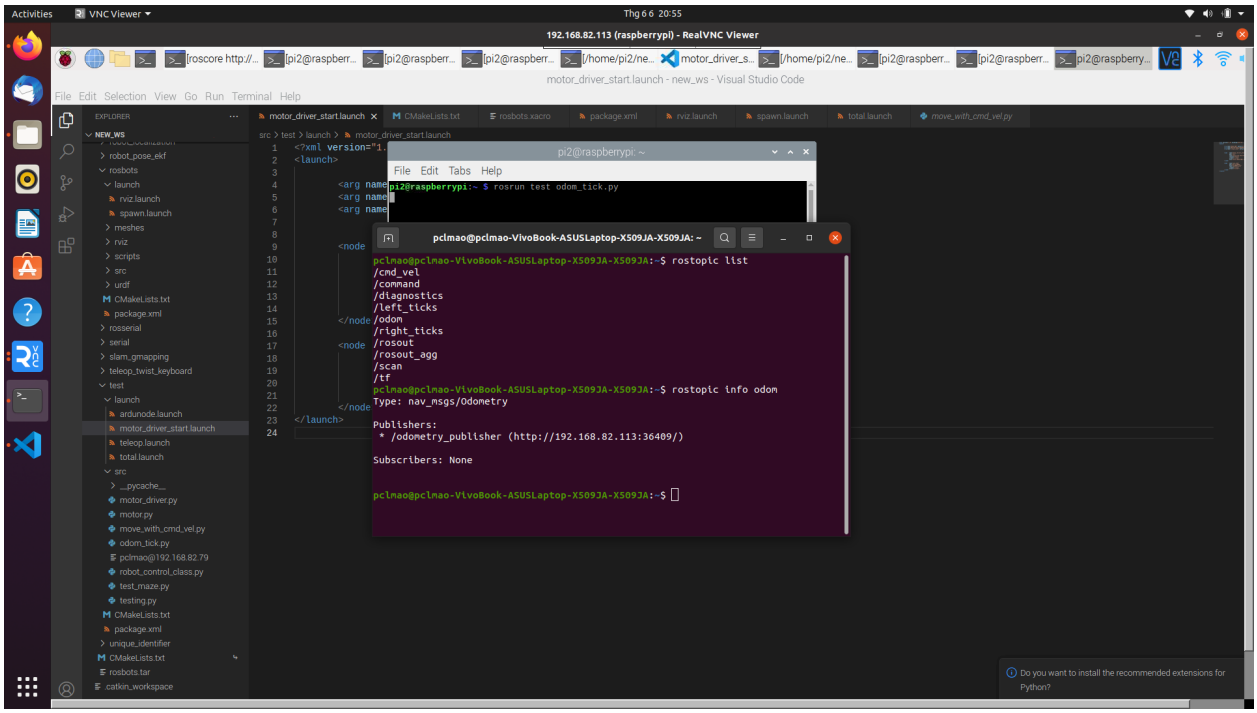
  odom.child_frame_id = "base_footprint"
  odom.twist.twist = Twist(Vector3(vx, vy, 0), Vector3(0, 0, vth))

  odom_pub.publish(odom)
```

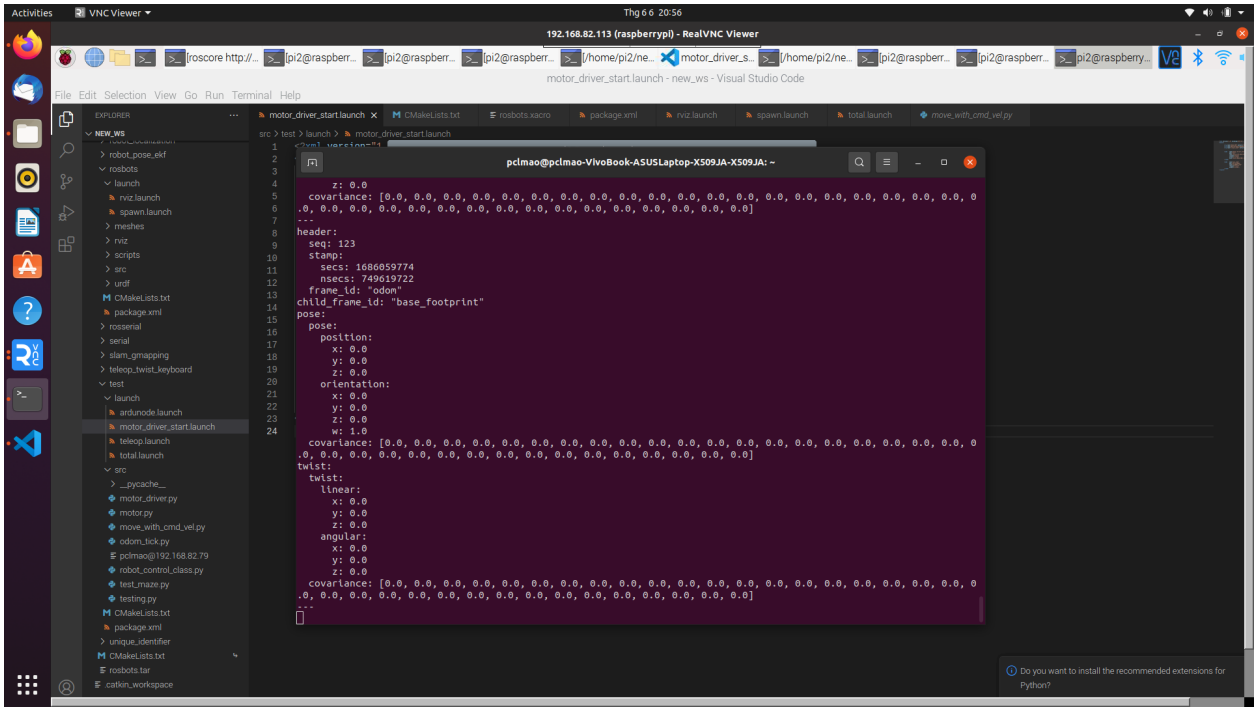
The purpose of doing that is to estimate the position and orientation. Odometry provides information about the robot's pose after a series of motions which is essential for localization, mapping, and navigation.

```
~$ rosrun test odom_tick.py
```

Run the above code on SBC and view the two new topics that it generates corresponding to each wheel **/left_ticks** and **/right_ticks**.



Here is the structure of an odom message :



By using odometry over Ros, we can use tools and libraries for robot development including mapping and localization packages(gmapping, AMCL,...), and visualization tools(Rviz,...) that can help us build our car. Usually, the car odometry is at the point that we start the SBC and publish **/odom** topic.

You can read more about Odometry at here.

2.5 Mapping

To navigate our robot autonomously, we need to use the ROS Navigation Stack. To launch the navigation stack, we can take a quick look at it:

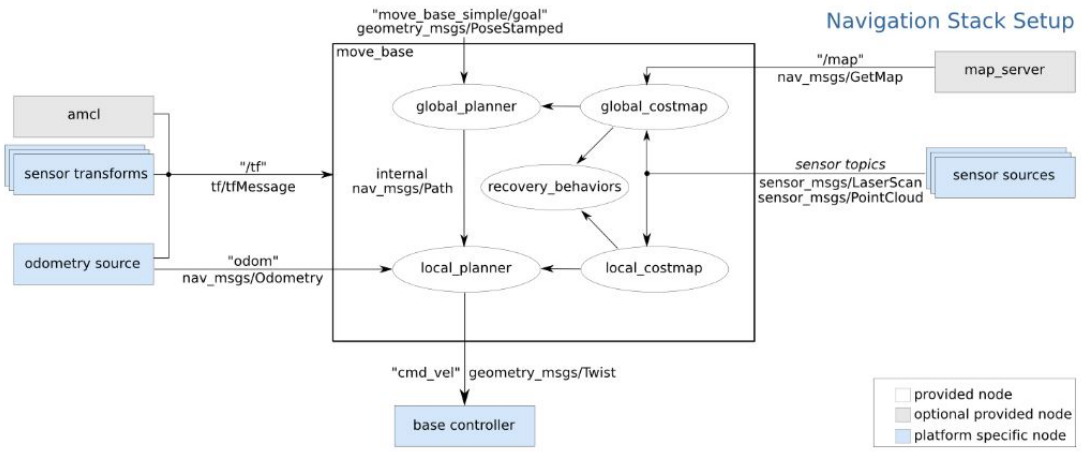


Fig.1.1 - Navigation Stack Setup (figure from ROS Wiki)

That's why we provided those topics in the above sections! Hence we will have to do 3 specific tasks to navigate our robot, that is Mapping, Localization, and Move Base.

2.5.1 Hector mapping

Hector SLAM (Simultaneous Localization and Mapping) is a popular open-source ROS (Robot Operating System) package used for mapping and localization in mobile robotics. It allows a robot to build a map of its environment while simultaneously estimating its own position within that map.

Hector SLAM is specifically designed for lightweight mobile robots that are equipped with a 2D laser scanner. It is often used in scenarios where accurate localization and mapping are required, such as autonomous exploration, navigation, and path planning.

The package utilizes a technique called scan matching to align the laser scans obtained from the robot's sensor with the previously constructed map. This process involves finding the best possible transformation (rotation and translation) that aligns the new scan with the map, minimizing the discrepancies between the two. By iteratively refining the robot's estimated pose, Hector SLAM can create an accurate and up-to-date representation of the environment.

Key features of the Hector SLAM ROS package include:

- Map Representation: Hector SLAM builds a 2D occupancy grid map that represents the robot's surroundings. The map indicates the likelihood of

each grid cell being occupied by obstacles or free space.

- Pose Estimation: The package estimates the robot's pose (position and orientation) by fusing information from the laser scanner and the odometry (motion) data of the robot. This fusion is performed using an Extended Kalman Filter (EKF) or an Unscented Kalman Filter (UKF). Although the package can function perfectly without odometry input.
- Loop Closure: Hector SLAM has the ability to detect loop closures, which occur when the robot revisits a previously explored area. Loop closures are essential for correcting accumulated errors in the map and improving the overall localization accuracy.
- Real-time Operation: The package is designed to work in real-time, allowing the robot to update its map and estimate its pose continuously as it explores the environment.

Hector SLAM is widely used within the ROS community and has been successfully implemented on various robotic platforms. It provides a reliable and efficient solution for mapping and localization, enabling robots to navigate and interact with their surroundings autonomously.

2.5.2 gmapping

ROS gmapping is a package in the Robot Operating System (ROS) that provides a method for constructing maps of an environment using sensor data from a robot. "gmapping"stands for "grid-based mapping."

The gmapping package uses a technique called Simultaneous Localization and Mapping (SLAM) to build maps while the robot simultaneously localizes itself within the map. SLAM is a fundamental capability in robotics that allows a robot to explore and navigate unknown environments autonomously.

The gmapping package specifically uses laser range finder data (typically from a 2D laser scanner) and odometry information from a robot to perform SLAM. It leverages the sensor readings and robot motion to estimate the robot's position and simultaneously build a grid-based map of the surrounding environment.

The gmapping algorithm applies probabilistic techniques to construct the map. It divides the environment into a grid of cells and assigns probabilities

to each cell to represent the presence or absence of obstacles. By continuously updating these probabilities based on sensor measurements and robot motion, gmapping creates an accurate map of the environment.

The resulting map can be used for various purposes, such as path planning, localization, and navigation. It enables robots to autonomously explore and understand their surroundings, which is crucial for tasks like autonomous mobile robotics, mapping, and exploration.

Overall, the ROS gmapping package is a powerful tool for creating maps of the environment and localizing a robot within those maps, facilitating autonomous navigation and intelligent decision-making for robotic systems.

2.5.3 Difference between Hector and gmapping

The main difference between the Hector SLAM package and the ROS gmapping package lies in their underlying mapping algorithms and their suitability for different robot configurations and environments.

1. Mapping Algorithm:
   - Hector SLAM: Hector SLAM is based on a scan matching approach, where it aligns laser scans with the previously constructed map to estimate the robot's pose and build the map. It uses an occupancy grid map representation.
   - gmapping: gmapping (Grid-based FastSLAM) is based on a particle filter approach, specifically the FastSLAM algorithm. It uses a grid-based map representation with a probabilistic occupancy grid to represent the environment.
2. Sensor Requirement:
   - Hector SLAM: Hector SLAM is specifically designed for 2D laser scanners. It assumes that the robot is equipped with a laser range finder (LIDAR) as its primary sensor. As such, it can function without the use of any other sensors for odometry.
   - gmapping: gmapping is more versatile and can work with various sensor configurations, including laser range finders, RGB-D cameras, or a combination of both. Unlike Hector, gmapping cannot function

without odometry data from other sensors on the robot.

3. Robot Configuration:
   - Hector SLAM: Hector SLAM is optimized for lightweight mobile robots, such as small ground robots or drones, that have limited computational resources.
   - gmapping: gmapping is more suitable for a wider range of robot configurations, including larger robots with more computational power.
4. Map Representation:
   - Hector SLAM: Hector SLAM uses a 2D occupancy grid map, which represents the environment as a grid of cells, indicating the likelihood of occupancy or free space in each cell.
   - gmapping: gmapping also uses a grid-based map representation, but it incorporates probabilistic occupancy values for each cell, allowing for uncertainty modeling.
5. Loop Closure Detection:
   - Hector SLAM: Hector SLAM has built-in loop closure detection capabilities, which help to correct accumulated errors in the map and improve localization accuracy.
   - gmapping: Loop closure detection is not a native feature of gmapping. However, it can be implemented by integrating additional techniques or using external libraries.

Considering these differences and our robot setups, we decided to go with Hector mapping as it is more suitable for our setup.

2.5.4 Mapping setup

Before localization, a map is needed as an input for the localization algorithm. Our mapping process involves initializing the Hector mapping node and manually operating our robot through the environment to obtain the local map. First we launch the file responsible for initializing the mapping node as well as starting Rviz.

```
~$ roslaunch hector_slam_launch tutorial.launch
```

Content of **tutorial.launch**:

```
<?xml version="1.0"?>

<launch>

  <arg name="geotiff_map_file_path" default="$(find hector_geotiff)/maps"/>

  <param name="/use_sim_time" value="false"/>

  <!-- <node pkg="rviz" type="rviz" name="rviz"
    args="-d $(find hector_slam_launch)/rviz_cfg/mapping_demo.rviz"/> -->

  <!-- Run Rviz -->
  <include file="$(find rosbots)/launch/rviz.launch"/>

  <include file="$(find hector_mapping)/launch/mapping_default.launch"/>

  <include file="$(find hector_geotiff_launch)/launch/geotiff_mapper.launch">
    <arg name="trajectory_source_frame_name" value="scanmatcher_frame"/>
    <arg name="map_file_path" value="$(arg geotiff_map_file_path)"/>
  </include>

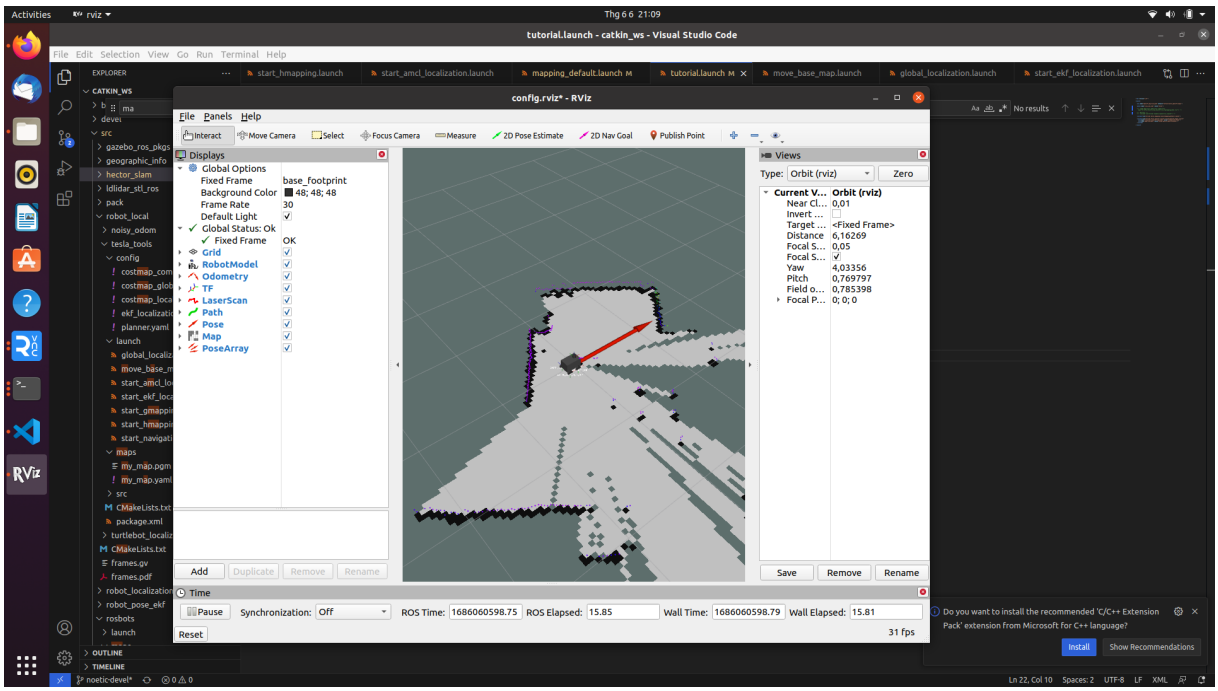
</launch>
```

Then we run teleop_twist_keyboard to control the robot.

```
~$ rosrun teleop_twist_keyboard teleop_twist_keyboard.py
```

As the robot traverses the environment, it will map the area accordingly. The resulting map is published to the **/map** topic and can be saved by calling the **map_saver** service.

```
~$ rosrun map_server map_saver -f my_map
```



2.6 Localization

2.6.1 Reinforcement Learning and Monte Carlo

Reinforcement Learning (RL) is a very distinct area of Machine Learning (ML), which defines the **agent** (in our case, the robot) and the **environment** (the area the agent acts). The environment is represented by **state** - a set of variables that completely describe the environment. Agents may or may not have access to the actual environment's state. However, the agent can observe something from the environment. The set of variables the agent perceives at any given time is called an **observation**.

The agent at every state interacts with the environment, performing a certain **action** (for example, moves to a certain position) – according to **policy** - which can be associated with the rule that the agent decides what kind of action to take. The policy can be denoted as an agent's brain. Normally, the number of actions that are available to the agent in each state is constant (but can vary). The set of all actions in all states is referred to as the **action space**.

The agent, which interacts with the environment (by perceiving the environment and performing certain actions) receives the sum of all **rewards**, which

is known as the **return**.

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \ldots + R_H$$

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots + \gamma^{H-1} R_H$$

The total return is expressed as:

$$G_t = \sum_{k=0}^{H} \gamma^k r_{t+k+1}$$

Where Y is the **discount factor** and H is the **horizon**, that can be infinite (discont factor will be discuss later)

The goal of the agent is to maximize its cumulative reward, called return. The agent RL learns behaviors (actions) to achieve this goal. This is the essence of RL. Generally speaking, all of the RL algorithms try to solve the given problem (or the environment) by maximizing the cumulative reward (return) the agent can collect.

Each action taken by the Agent influences the environment. The environment responds to the agent's action and transits from one state to the other. The function that is responsible for this transition is called the **transition function**.

The actions, states, transitions, and rewards determine the Markov Decision Processes (MDP), a mathematical framework for RL. Typically, in RL, we assume all environments are framed as a MDP. This concept will be discussed later in this section.

2.6.2 AMCL algorithm

The ROS AMCL (Adaptive Monte Carlo Localization) package is a widely used package in the Robot Operating System (ROS) ecosystem. It provides a probabilistic localization system for autonomous robots, allowing them to estimate their own position and orientation within an environment.

AMCL is based on the Monte Carlo Localization (MCL) algorithm, which uses a particle filter to represent the belief state of the robot's pose. The algorithm takes in sensor measurements, such as laser scans or odometry

data, and uses them to update the belief state by resampling particles. Over time, as the robot receives more sensor data, the particles converge towards the true pose of the robot.

The AMCL package in ROS provides a convenient implementation of the MCL algorithm, along with various features to improve localization accuracy. It incorporates sensor models to handle different types of sensors, supports different coordinate frames, and allows for map-based localization by using a pre-built map of the environment.

By using the AMCL package, we can achieve accurate and robust localization for our robot, enabling it to perform tasks such as navigation, mapping, and exploration in a variety of environments.

2.6.3 AMCL setup

The setup is in a similar manner to launching Hector mapping, through a prepared launch file named **global_localization** which includes the following:

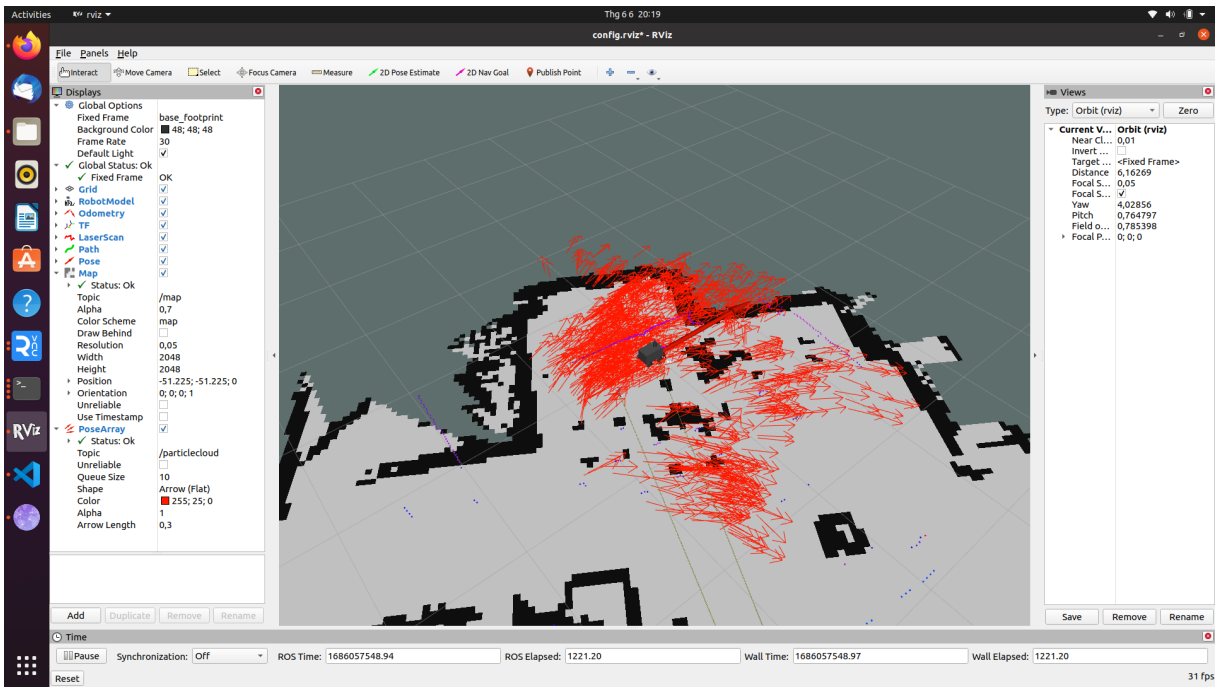
```
<launch>

<!--- Start AMCL Localization -->
<include file="$(find tesla_tools)/launch/start_amcl_localization.launch"/>

<!-- Start EKF Localization -->
<include file="$(find tesla_tools)/launch/start_ekf_localization.launch" />

</launch>
```

The launch file starts AMCL node as expected, while also launching another package called **robot_pose_ekf** which helps the AMCL algorithm perform better by filtering the odom data obtained from **/odom** topic. The resulting localization (in progress):



2.7 Move Base

You can read more here

2.8 Our Code

https://github.com/Quoc1610/Multidisciplinary-Project---HK222?fbclid=IwAR30rOi24ndjGvaLdujeCDlTtIieBJtNeO7ZXkGYGDxK-Ltj-S-idvf0ahw

REFERENCES

[1] Robotignight Academy. ”Building and Rosifying a Robot from Scratch - Chapter 4: Creating the motor drivers.”

[2] Elecrow. “LiDAR Sensor LD19 Development Manual [2022].” Elecrow Post, 15 Mar. 2022, https://www.elecrow.com/download/product/SLD06360F/LD19_Development%20Manual_V2.3.pdf